Which Verification Strategy Makes Multi-Agent Financial Advisory Systems More Reliable?

Bhoomika Monthy Rajashekar **Faisal Alsabih** **Tejas Vanjari** **Dhanya Kini Bailoor**
Northeastern University

Abstract : LLM agents can draft wealth and financial advice, but their answers may contain confident arithmetic or factual errors that a client would act on. We study verification, the step that decides whether such an answer should be trusted. We build a multi-agent financial advisory pipeline and compare four conditions on 10 to 20 objectively verifiable tasks over synthetic client data: no verification, self-verification, peer verification, and a dedicated verifier. Ground truth is computed deterministically in code. We measure task accuracy, error detection, false corrections, token cost, and latency to learn which strategy most improves reliability, and at what price.

Keywords : multi-agent systems, LLM verification, financial advisory, reliability

1 PROBLEM AND MOTIVATION

Agents built on large language models are increasingly used to draft wealth and financial advice: how far a portfolio has drifted from its target allocation, what a savings plan grows to under fees and contributions, whether a product fits a client's risk profile. Their answers read confidently even when the arithmetic or the facts behind them are wrong, and an unchecked error in financial advice can cost a client real money. Such an agent is easy to build and hard to trust, so the bottleneck is verification.

2 RESEARCH QUESTION

Which verification strategy most improves the reliability of a multi-agent financial advisory system?

3 SYSTEM AND APPROACH

An advisor agent receives a client profile (age, goals, risk tolerance, holdings, fees) and a question, and returns a recommendation together with the numbers and rules it relied on. A verification layer checks that output before it is accepted (Fig. 1). We compare the four conditions in Table 1 on the same task suite. To see how much the result depends on verifier quality, we also run a deliberately weakened verifier. No verifier sees the answer key.

Table 1: Verification conditions under comparison.

Condition	Checked by	What it does
No verification	none	First answer is final (baseline)
Self-verification	advisor itself	Re-checks its own answer
Peer verification	second advisor	Independently checks or re-derives
Dedicated verifier	checker agent	Built only to check; has a calculator tool

4 DATA AND GROUND TRUTH

Synthetic data is sufficient for this study and has a practical advantage: because we generate it, every task has a known correct answer. We programmatically create client profiles, portfolios, and market scenarios, and from them 10 to 20 objectively verifiable tasks (allocation drift, projected balances, rebalancing amounts, suitability checks against written policy rules). Ground truth is computed deterministically in code and hidden from all agents.

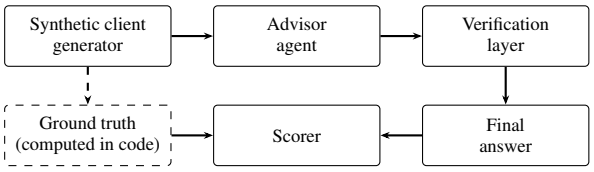


Figure 1: Pipeline. The generator produces tasks with known answers; the scorer compares each final answer to ground truth. Agents never see the ground truth.

5 EVALUATION

The primary metric is task accuracy. Secondary metrics are error detection (share of wrong answers flagged), false corrections (correct answers wrongly changed), token cost, and latency. Each task runs several times per condition to measure variance, and failures are labeled by type so we can report which strategy catches which kind of error.

6 DELIVERABLES AND ETHICS

We will deliver a modular, unit-tested Python pipeline (data generator, agents, evaluation harness) in a public GitHub repository, and a report comparing the conditions on accuracy against cost and latency. The main ethical risk is over-reliance on automated financial advice. We mitigate it by presenting outputs as decision support rather than advice, letting the verifier abstain and flag a human review when it cannot confirm a claim, and stating that results come from synthetic scenarios and make no claims about real clients. Synthetic data also avoids handling real client information.

7 MILESTONES

We will follow the course schedule: proposal with architecture and method (Oct 21); progress report with the data pipeline and baseline results (Nov 18); final report, presentation, and code (Dec 9).